

DATA STRUCTURES AND ALGORITHM

23CSE203

LAB MANUAL

SUBMITTED BY:	
NAME	**S.Mohitha**
ROLL NO	**AV.SC.U4CSE25039**
YEAR/SEM/SECTION	**2nd/3rd/CSE A**
CREDITS	**05**

SUBMITTED TO:	
NAME	**DR. RAJ KUMAR BATCHU**
DESIGNATION	**ASST.PROFESSOR (SI.Grade)**
DEPARTMENT	**SCHOOL OF COMPUTING**

INDEX

S.NO	PROGRAM	PAGE NO
1	**WEEK1:** **1(A). A rocket launch displays a countdown before launch. Instead of** **using a loop, implement the countdown using recursion. Write a recursive Python program to display a countdown from n to 1, followed** **by "Launch!".** **1(B). Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p^n (power) where:** **□ p = principal growth factor** **□ n = number of years** **1(C). An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.** **1(D). In a logistics company, the number of possible ways to arrange n** **different parcels are calculated using the factorial formula (n!). Develop** **a recursive program to calculate the factorial of a given number.** **1(E). Write a recursive Python program to print the first n terms of Fibonacci series**	
2	**WEEK2:** **2(A). Write a Python program to implement linear search.** **2(B) i. Write a Python program to implement binary search with sorted** **array as an input.** **2(B) ii. Write a Python program to implement binary search without sorted array as an input**	
3	**WEEK 3:** **3(A).Write a python program to implement bubble sort where the input** **elements are taken from the user** **3(B). Write a python program to implement insertion sort where the** **input elements are taken from the user** **3(C). Write a python program to implement selection sort where the** **input elements are taken from the user**	

WEEK – 1

1)AIM: A rocket launch system displays a countdown before launch. Instead of using a loop implement the countdown using recursion .write a recursion python program to display a countdown from n to 1 followed by “launch”.

PROGRAM:

```
def launch(n):
    for i in range(n,-1,-1):
        if i==0:
            print("launch")
        else:
            print(i)
n=int(input("enter no. of seconds to launch:"))
launch(n)
```

OUTPUT:

```
>>>
============== RESTART: C:/Users/mohit/OneDrive/Desktop/lab1_1.py ==============
enter no. of seconds to launch:7
7
6
5
4
3
2
1
launch
>>>
```

EXPLANATION:

This program displays a countdown from n to 1 and finally prints 'launch'. Although the aim mentions recursion, the given code uses a loop.

ERROR TABLE:

Error	Cause	Solution
Indentation Error	Incorrect indentation	Align blocks properly
Name Error	Variable not defined	Define before use
Value Error	Non-integer input	Enter valid integer

2)AIM: banks calculate the growth of an investment using compoundinterest the formula requires calculating powers. Which can be implemented recursively. Write a recursive program to calculate p^n(power),where p=principal growth factor, n= number of years.

PROGRAM:

```
def intrest(p,n):
    if n==0:
        return 1
    else:
        return p*intrest(p,n-1)
n=int(input("enter no.of years:"))
p=int(input("enter a number:"))
res=intrest(p,n)
print(res)
```

OUTPUT:

```
========== RESTART: C:/Users/mohit/OneDrive/Desktop/lab1_2.py ==========
enter no.of years:4
enter a number:25
390625
>>>
```

EXPLANATION:

This program recursively calculates p^n by multiplying p until n becomes 0.

ERROR TABLE:

Error	Cause	Solution
Recursion error	Very large value of n	Reduce input
Value Error	Non-integer input	Enter valid integer

3)AIM: An HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID .write a recursive function to search for an employee ID in a list.

PROGRAM:

```
def search(m):
    key=m
    for i in range(n-1):
```

```
        if arr[i]==key:
            return i
n=int(input("enter no.of employees to add:"))
arr=[]
for i in range(n):
    a=int(input("enter a value:"))
    arr.append(a)
print(arr)
m=int(input("enter a element to search:"))
result=search(m)
if result==-1:
    print(“element is not found”)
else:
    print("employee id is stored at the index:",result)
```

OUTPUT:

```
>>>
==================== RESTART: C:/Users/mohit/OneDrive/Desktop/lab1_3.py ====================
enter no.of employees to add:4
enter a value:11
enter a value:12
enter a value:16
enter a value:17
[11, 12, 16, 17]
enter a element to search:12
employee id is stored at the index: 1
```

EXPLANATION:

This program searches for an employee ID in a list and returns its index if found.

ERROR TABLE:

Error	Cause	Solution
Syntax error	Wrong quotes/brackets	Correct syntax
Logic error	Returns None instead of -1	Return -1 when not found

4)AIM: In a logistics company the number of possible ways to arrange n different parcels is calculated using the factorial formula(n!). develop a recursive program to calculate the factorial of a given number.

PROGRAM:

```
def fact(n):
    fact=1
    for i in range(1,n+1):
        fact=fact*i
    return fact
n=int(input("enter a number to find its factorial:"))
result=fact(n)
print(result)
```

OUTPUT:

```
==================== RESTART: C:/Users/mohit/OneDrive/Desktop/lab1_4.py ====================
enter a number to find its factorial:12
479001600
>>>
```

EXPLANATION:

This program calculates the factorial of a number using iteration (not recursion).

ERROR TABLE:

Error	Cause	Solution
Overflow/large input	Large input value	Give smaller input
Value error	Invalid input	Even integer

5)AIM: write a recursive python program to print the first n terms of the Fibonacci series.

PROGRAM:

```
def fib(n):
    if n==0:
        return 0
    elif n==1:
        return 1
```

```
    else:
        return fib(n-1)+fib(n-2)
n=int(input("enter no of terms:"))
if n<=-1:
   print("invalid input")
else:
   for i in range(n):
      print(fib(i),end=" ")
```

OUTPUT:

```
============== RESTART: C:/Users/mohit/OneDrive/Desktop/lab1_5.py ==============
enter no of terms:5
0 1 1 2 3
>>>
```

EXPLANATION:

The program uses recursion to generate Fibonacci numbers. fib (0)=0 and fib (1) =1 are the base cases. For every other term, fib(n)=fib(n-1) + fib(n-2). The loop prints the first n terms.

ERROR TABLE:

Error	Cause	Solution
Recursion error	Large value of n	Use iteration
Invalid input	Negative value of n	Validate input
Slow execution	Repeated recursive calls	Use dynamic programming

WEEK – 2

2(A). AIM:Write a Python program to implement linear search.

PROGRAM:

```
def linear_search(arr,key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1
n = int(input("Enter the number of elements:"))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
key = int(input("Enter the element to search:"))
result = linear_search(arr,key)
if result!=1:
    print("Element found at index",result)
else:
    print("Element not found")
```

OUTPUT:

```
>>>
=============== RESTART: C:/Users/mohit/OneDrive/Desktop/lab2A.py ==============
Enter the number of elements:4
Enter the elements:
11
14
9
23
Enter the element to search:23
Element found at index 3
>>>
```

EXPLANATION:

The linear_search() function checks each element of the list one by one and compares it with the given key.

If the key is found, its index is returned; otherwise, the function returns -1.

The program takes the array and search element as input and displays whether the element is found.

ERROR TABLE:

S NO	ERRORS	RECTIFICATION
	ValueError occurs if a non-integer value is entered when int(input()) is used.	Enter only integer values for the number of elements and array elements.
	TypeError may occur if len(arr) is used on a variable that is not a list.	Make sure arr is initialized as a list using arr = [] and elements are appended to it.

2(B) i.AIM: Write a Python program to implement binary search with sorted array as an input.

PROGRAM:

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

n = int(input("Enter the number of elements:"))
arr = []

print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
```

```
key = int(input("Enter the element to search:"))
result = linear_search(arr, key)


if result != -1:
    print("Element found at index", result)
else:
    print("Element not found")
```

OUTPUT:

POSITIVE OUTPUT:

```
>>>
=============== RESTART: C:/Users/mohit/OneDrive/Desktop/lab2B.py ==============
Enter the number of elements:6
Enter the elements:
11
23
35
45
56
67
Enter the element to search:23
Element found at index 1
```

NEGATIVE OUTPUT:

```
>>>
=============== RESTART: C:/Users/mohit/OneDrive/Desktop/lab2B.py ==============
Enter the number of elements:6
Enter the elements:
11
23
35
45
56
67
Enter the element to search:87
Element not found
```

EXPLANATION:

1. The program takes a sorted array as input and searches for the required element using binary search.

13

2. It checks the middle element and eliminates either the left or right half based on the comparison.

3. If the element is found, its position is displayed; otherwise, it prints “Element not found.”

S.NO	ERRORS	Rectification
1.	Binary search may give an incorrect result if the input array is unsorted.	Check whether the array is sorted and use arr.sort() before calling binary_search().
2.	IndexError can occur if mid is calculated or the low/high boundaries are modified incorrectly	Initialize low = 0, high = len(arr) - 1 and update them as low = mid + 1 or high = mid - 1.

2(B) ii.AIM: Write a Python program to implement binary search without sorted array as an input

PROGRAM:

```
def binary_search(arr,target):
    arr.sort()
    left = 0
    right = len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
n = int(input("Enter the number of elements:"))
arr = []
print("Enter the elements:")
for i in range(n):
```

```
    arr.append(int(input()))
target = int(input("Enter the element to search:"))
index = binary_search(arr,target)
print("Sorted array:",arr)
if index != -1:
    print(f"Element found at index {index} in the sorted array.")
else:
    print("Element not found.")
```

OUTPUT:

POSITIVE OUTPUT:

```
>>>
============== RESTART: C:/Users/mohit/OneDrive/Desktop/lab2Bii.py ============
Enter the number of elements:5
Enter the elements:
23
67
98
37
18
Enter the element to search:37
Sorted array: [18, 23, 37, 67, 98]
Element found at index 2 in the sorted array.
```

NEGATIVE OUTPUT:

```
>>>
============== RESTART: C:/Users/mohit/OneDrive/Desktop/lab2Bii.py =============
Enter the number of elements:5
Enter the elements:
23
67
98
37
18
Enter the element to search:64
Sorted array: [18, 23, 37, 67, 98]
Element not found.
```

EXPLANATION:

1. The program takes an unsorted array as input and first sorts it using arr.sort().

2. It then uses binary search by checking the middle element and reducing the search range.

3. If the element is found, its position is displayed; otherwise, it prints “Element not found.”

ERROR TABLE:

S.NO.	ERRORS	RECTIFICATION
1.	Logical Error	Use a found variable and print “Element not found” if it remains False.
2.	Logical Error	Use mid = (low + high) // 2 to calculate the middle index.

WEEK-3

3A.

AIM:Write a python program to implement bubble sort where the input elements are taken from the user

PROGRAM:

```
n = int(input("Enter the number of elements: "))


arr = []


for i in range(n):
    element = int(input(f"Enter element {i + 1}: "))
    arr.append(element)


for i in range(n - 1):
    for j in range(n - 1 - i):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]


print("Sorted elements:", arr)
```

OUTPUT:

```
>>>
============== RESTART: C:/Users/mohit/OneDrive/Desktop/lab3_1.py ==============
Enter the number of elements: 4
Enter element 1: 12
Enter element 2: 45
Enter element 3: 23
Enter element 4: 76
Sorted elements: [12, 23, 45, 76]
>>>
```

EXPLANATION:

1. The program takes the number of elements and their values from the user and stores them in an array.

2. Bubble Sort compares adjacent elements and swaps them if they are in the wrong

order.

3. After all passes, the array is sorted in ascending order and displayed.

ERROR TABLE:

S.NO	ERRORS	RECTIFICATION
1	Name Error	Initialize arr = [] before taking input
2	Value Error	Enter only integer values

3(B).AIM: Write a python program to implement insertion sort where the input elements are

taken from the user

PROGRAM:

```
n = int(input("Enter the number of elements: "))


arr = []


for i in range(n):
    element = int(input(f"Enter element {i + 1}: "))
    arr.append(element)


for i in range(1, n):
    key = arr[i]
    j = i - 1


    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j -= 1


    arr[j + 1] = key
```

print("Sorted elements:", arr)

OUTPUT:

```
============== RESTART: C:/Users/mohit/OneDrive/Desktop/LAB3_2.py ==============
Enter the number of elements: 5
Enter element 1: 12
Enter element 2: 32
Enter element 3: 45
Enter element 4: 21
Enter element 5: 97
Sorted elements: [12, 21, 32, 45, 97]
>>>
```

EXPLANATION:

1.The program takes the number of elements and their values from the user and stores them

in an array.

2.Insertion Sort selects each element and inserts it into its correct position among the

previously sorted elements.

3. Finally, the sorted array is displayed in ascending order.

ERROR TABLE:

S.NO.	ERRORS	RECTIFICATION
1	SyntaxError	Use insertion_sort with an underscore.
2	Name Error	Call insertion_sort(arr) correctly.

3(C).AIM: Write a python program to implement selection sort where the input elements are

taken from the user

PROGRAM:

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2


        left = arr[:mid]
        right = arr[mid:]
```

```
    merge_sort(left)
    merge_sort(right)

    i = 0
    j = 0
    k = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            arr[k] = left[i]
            i += 1
        else:
            arr[k] = right[j]
            j += 1
        k += 1

    while i < len(left):
        arr[k] = left[i]
        i += 1
        k += 1

    while j < len(right):
        arr[k] = right[j]
        j += 1
        k += 1

n = int(input("Enter the number of elements: "))

arr = []
```

```
for i in range(n):
    element = int(input(f"Enter element {i + 1}: "))
    arr.append(element)

merge_sort(arr)

print("Sorted elements:", arr)
```

OUTPUT:

```
>>>
============== RESTART: C:/Users/mohit/OneDrive/Desktop/LAB3_3.py ==============
Enter the number of elements: 4
Enter element 1: 23
Enter element 2: 12
Enter element 3: 76
Enter element 4: 34
Sorted elements: [12, 23, 34, 76]
>>>
```

EXPLANATION:

1. The program takes the number of elements and their values from the user and stores them in an array.

2. Selection Sort finds the smallest element and swaps it with the first unsorted element.

3. This process continues until the array is completely sorted in ascending order.

ERROR TABLE:

S.NO.	ERRORS	RECTIFICATION
1	NameError	Initialize arr = [] before taking input.
2	Value Error	Enter only integer values

3(d).AIM:Write a python program to implement quick sort where the input elements are taken from the user

PROGRAM:

```
def quick_sort(arr):

    if len(arr) <= 1:
        return arr
```

```
    pivot = arr[-1]

    left = []
    right = []

    for i in arr[:-1]:
        if i < pivot:
            left.append(i)
        else:
            right.append(i)

    return quick_sort(left) + [pivot] + quick_sort(right)

n = int(input("Enter the number of elements:"))

arr = []

for i in range(n):
    element = int(input(f"Enter element{i + 1}:"))
    arr.append(element)

arr = quick_sort(arr)

print("Sorted elements:", arr)
```

OUTPUT:

```
>>>
============== RESTART: C:/Users/mohit/OneDrive/Desktop/LAB3_4.py ==============
Enter the number of elements:4
Enter element1:23
Enter element2:12
Enter element3:65
Enter element4:87
Sorted elements: [12, 23, 65, 87]
>>>
```

3E.AIM:Write a python program to implement selection sort where the input elements are taken from the user

PROGRAM:

```
n = int(input("Enter the number of elements: "))


arr = []


for i in range(n):
    element = int(input(f"Enter element {i + 1}: "))
    arr.append(element)


for i in range(n - 1):
    min_index = i


    for j in range(i + 1, n):
        if arr[j] < arr[min_index]:
            min_index = j


    arr[i], arr[min_index] = arr[min_index], arr[i]


print("Sorted elements:", arr)
```

```
>>>
============== RESTART: C:/Users/mohit/OneDrive/Desktop/LAB3_5.py ==============
Enter the number of elements: 6
Enter element 1: 98
Enter element 2: 68
Enter element 3: 45
Enter element 4: 65
Enter element 5: 23
Enter element 6: 16
Sorted elements: [16, 23, 45, 65, 68, 98]
>>>
```

EXPLANATION:

1. The program takes the number of elements and their values from the user and stores them in an array.

2. Selection Sort finds the smallest element and swaps it with the first unsorted element.

3. This process continues until the array is completely sorted in ascending order.

ERROR TABLE:

S.NO.	ERRORS	RECTIFICATION
1.	TypeError	Convert input values using int(input())..
2.	Logic Error	Update min_index correctly and swap arr[i] with arr[min_index].

4A. AIM: Write a program to implement a single linked list which would perform the below operations, by taking an input from the user.

a)create a linked list

b)insert at beginning

c)insert at end

d)insert at a specified index

e)delete by value

f)delete 1st node

g)delete last node

h)count no.of nodes

i)display or traverse

j)search for a number

k)exit

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None



class LinkedList:
    def __init__(self):
        self.head = None

    def create(self):
        n = int(input("Enter number of elements: "))

        for i in range(n):
            data = int(input("Enter data: "))
```

```
        new = Node(data)

        if self.head is None:
            self.head = new
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new

    def insert_begin(self, data):
        new = Node(data)
        new.next = self.head
        self.head = new

    def insert_end(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new

    def insert_index(self, index, data):
        if index < 0:
            print("Invalid index")
```

```
            return

        if index == 0:
            self.insert_begin(data)
            return

        temp = self.head

        for i in range(index - 1):
            if temp is None:
                print("Invalid index")
                return
            temp = temp.next

        if temp is None:
            print("Invalid index")
            return

        new = Node(data)
        new.next = temp.next
        temp.next = new

    def delete_value(self, data):
        if self.head is None:
            print("Linked list is empty")
            return

        if self.head.data == data:
            self.head = self.head.next
```

```
            return

        temp = self.head

        while temp.next:
            if temp.next.data == data:
                temp.next = temp.next.next
                return
            temp = temp.next

        print("Value not found")

    def delete_first(self):
        if self.head is None:
            print("Linked list is empty")
        else:
            self.head = self.head.next

    def delete_last(self):
        if self.head is None:
            print("Linked list is empty")
            return

        if self.head.next is None:
            self.head = None
            return

        temp = self.head
```

```
        while temp.next.next:
            temp = temp.next

        temp.next = None

    def count(self):
        count = 0
        temp = self.head

        while temp:
            count = count + 1
            temp = temp.next

        print("Number of nodes:", count)

    def display(self):
        temp = self.head

        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next

        print("None")


l = LinkedList()

l.create()
print("Linked list:", end=" ")
```

```
l.display()

data = int(input("Enter data to insert at beginning: "))
l.insert_begin(data)
print("After insertion:", end=" ")
l.display()

data = int(input("Enter data to insert at end: "))
l.insert_end(data)
print("After insertion:", end=" ")
l.display()

index = int(input("Enter index: "))
data = int(input("Enter data: "))
l.insert_index(index, data)
print("After insertion:", end=" ")
l.display()

data = int(input("Enter value to delete: "))
l.delete_value(data)
print("After deletion:", end=" ")
l.display()

l.delete_first()
print("After deleting first node:", end=" ")
l.display()

l.delete_last()
print("After deleting last node:", end=" ")
```

l.display()

l.count()

OUTPUT:

```
>>>
============== RESTART: C:/Users/mohit/OneDrive/Desktop/lab4_1.py ==============
Enter number of elements: 5
Enter data: 22
Enter data: 12
Enter data: 54
Enter data: 87
Enter data: 67
Linked list: 22 -> 12 -> 54 -> 87 -> 67 -> None
Enter data to insert at beginning: 12
After insertion: 12 -> 22 -> 12 -> 54 -> 87 -> 67 -> None
Enter data to insert at end: 54
After insertion: 12 -> 22 -> 12 -> 54 -> 87 -> 67 -> 54 -> None
Enter index: 3
Enter data: 67
After insertion: 12 -> 22 -> 12 -> 67 -> 54 -> 87 -> 67 -> 54 -> None
Enter value to delete: 22
After deletion: 12 -> 12 -> 67 -> 54 -> 87 -> 67 -> 54 -> None
After deleting first node: 12 -> 67 -> 54 -> 87 -> 67 -> 54 -> None
After deleting last node: 12 -> 67 -> 54 -> 87 -> 67 -> None
Number of nodes: 5
>>>
```

EXPLANATION:

The LinkedList program creates and performs different operations on a singly linked list.

The create() function creates the linked list, while insert_begin(), insert_end(), and insert_index() insert new nodes at the beginning, end, and specified index.

The delete_value(), delete_first(), and delete_last() functions delete nodes from the list. The count() function counts the number of nodes, and the display() function traverses and displays all the elements.

In a singly linked list, each node contains two parts: data and next. The data stores the value, while next stores the reference to the next node. The next pointer of the last node is always None.

S NO	ERRORS	RECTIFICATION
1	ValueError occurs if a non-integer value is entered when int(input()) is used.	Enter only integer values for the number of elements, index, and node data.

S NO	ERRORS	RECTIFICATION
2	IndentationError may occur if statements inside classes, functions, if, else, while, or for blocks are not properly indented.	Maintain proper indentation throughout the program.
3	NameError may occur if variables such as temp, new, or data are misspelled or used before declaration.	Check variable names and initialize them before use.

4B.AIM: Write a program to implement a double linked list which would perform the below operations, by taking an input from the user.

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.prev = None
        self.data = data
        self.next = None



class DoublyLinkedList:
    def __init__(self):
        self.head = None


    # 1. Create linked list
    def create(self):
        n = int(input("Enter number of elements: "))


        for i in range(n):
            data = int(input("Enter data: "))
            new = Node(data)
```

```
        if self.head is None:
            self.head = new
        else:
            temp = self.head

            while temp.next:
                temp = temp.next

            temp.next = new
            new.prev = temp

    # 2. Insert at beginning
    def insert_begin(self, data):
        new = Node(data)

        new.next = self.head

        if self.head is not None:
            self.head.prev = new

        self.head = new

    # 3. Insert at end
    def insert_end(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
```

```
    else:
        temp = self.head

        while temp.next:
            temp = temp.next

        temp.next = new
        new.prev = temp

# 4. Insert at specific index
def insert_index(self, index, data):
    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        self.insert_begin(data)
        return

    temp = self.head

    for i in range(index - 1):
        if temp is None:
            print("Invalid index")
            return
        temp = temp.next

    if temp is None:
        print("Invalid index")
```

```
        return

    new = Node(data)

    new.next = temp.next
    new.prev = temp

    if temp.next is not None:
        temp.next.prev = new

    temp.next = new

# 5. Delete by value
def delete_value(self, data):
    if self.head is None:
        print("Linked list is empty")
        return

    temp = self.head

    while temp:
        if temp.data == data:

            if temp.prev is not None:
                temp.prev.next = temp.next
            else:
                self.head = temp.next

            if temp.next is not None:
```

```
            temp.next.prev = temp.prev

            return

        temp = temp.next

    print("Value not found")

# 6. Delete first node
def delete_first(self):
    if self.head is None:
        print("Linked list is empty")
        return

    self.head = self.head.next

    if self.head is not None:
        self.head.prev = None

# 7. Delete last node
def delete_last(self):
    if self.head is None:
        print("Linked list is empty")
        return

    temp = self.head

    while temp.next:
        temp = temp.next
```

```
        if temp.prev is None:
            self.head = None
        else:
            temp.prev.next = None

    # 8. Count number of nodes
    def count(self):
        count = 0
        temp = self.head

        while temp:
            count = count + 1
            temp = temp.next

        print("Number of nodes:", count)

    # 9. Display / Traverse
    def display(self):
        temp = self.head

        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.next

        print("None")



l = DoublyLinkedList()
```

```
l.create()
print("Linked list:", end=" ")
l.display()

data = int(input("Enter data to insert at beginning: "))
l.insert_begin(data)
print("After insertion:", end=" ")
l.display()

data = int(input("Enter data to insert at end: "))
l.insert_end(data)
print("After insertion:", end=" ")
l.display()

index = int(input("Enter index: "))
data = int(input("Enter data: "))
l.insert_index(index, data)
print("After insertion:", end=" ")
l.display()

data = int(input("Enter value to delete: "))
l.delete_value(data)
print("After deletion:", end=" ")
l.display()

l.delete_first()
print("After deleting first node:", end=" ")
l.display()
```

```
l.delete_last()
print("After deleting last node:", end=" ")
l.display()

l.count()
```

OUTPUT:

```
================================================================== RESTART: C:/Users/mohit/OneDrive/Desktop/lab4_2.py ==============
Enter number of elements: 6
Enter data: 34
Enter data: 98
Enter data: 56
Enter data: 15
Enter data: 72
Enter data: 45
Linked list: 34 <-> 98 <-> 56 <-> 15 <-> 72 <-> 45 <-> None
Enter data to insert at beginning: 56
After insertion: 56 <-> 34 <-> 98 <-> 56 <-> 15 <-> 72 <-> 45 <-> None
Enter data to insert at end: 72
After insertion: 56 <-> 34 <-> 98 <-> 56 <-> 15 <-> 72 <-> 45 <-> 72 <-> None
Enter index: 4
Enter data: 61
After insertion: 56 <-> 34 <-> 98 <-> 56 <-> 61 <-> 15 <-> 72 <-> 45 <-> 72 <-> None
Enter value to delete: 45
After deletion: 56 <-> 34 <-> 98 <-> 56 <-> 61 <-> 15 <-> 72 <-> 72 <-> None
After deleting first node: 34 <-> 98 <-> 56 <-> 61 <-> 15 <-> 72 <-> 72 <-> None
After deleting last node: 34 <-> 98 <-> 56 <-> 61 <-> 15 <-> 72 <-> None
Number of nodes: 6
>>>
```

EXPLANATION:

The DoublyLinkedList program creates and performs different operations on a doubly linked list.

The create() function creates the linked list, while insert_begin(), insert_end(), and insert_index() insert new nodes at the beginning, end, and specified index.

The delete_value(), delete_first(), and delete_last() functions delete nodes from the list. The count() function counts the number of nodes, and the display() function displays all the elements.

In a doubly linked list, each node contains **three parts: prev, data, and next**. The prev pointer points to the previous node, while the next pointer points to the next node.

S NO	ERRORS	RECTIFICATION
1	ValueError occurs if a non-integer value is entered when int(input()) is used.	Enter only integer values for the number of elements, index, and node data.
2	IndentationError may occur if statements inside classes, functions, if, else, while, or for blocks are not properly indented.	Maintain proper indentation throughout the program.

S NO	ERRORS	RECTIFICATION
3	AttributeError may occur if an incorrect attribute such as data, prev, or next is used.	Make sure self.data, self.prev, and self.next are used correctly.
4	NameError may occur if variables such as temp, new, or data are misspelled or used before declaration.	Check variable names and initialize them before use.

4C.AIM: **Write a program to implement a circular linked list which would perform the below operations, by taking an input from the user.**

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularLinkedList:
    def __init__(self):
        self.head = None

    # 1. Create linked list
    def create(self):
        n = int(input("Enter number of elements: "))

        for i in range(n):
            data = int(input("Enter data: "))
            new = Node(data)

            if self.head is None:
```

```
            self.head = new
            new.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            temp.next = new
            new.next = self.head

# 2. Insert at beginning
def insert_begin(self, data):
    new = Node(data)

    if self.head is None:
        self.head = new
        new.next = self.head
    else:
        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        new.next = self.head
        self.head = new
        temp.next = self.head

# 3. Insert at end
```

```
def insert_end(self, data):
    new = Node(data)

    if self.head is None:
        self.head = new
        new.next = self.head
    else:
        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        temp.next = new
        new.next = self.head

# 4. Insert at specific index
def insert_index(self, index, data):
    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        self.insert_begin(data)
        return

    if self.head is None:
        print("Invalid index")
        return
```

```
    temp = self.head

    for i in range(index - 1):
        temp = temp.next

        if temp == self.head:
            print("Invalid index")
            return

    new = Node(data)
    new.next = temp.next
    temp.next = new

# 5. Delete by value
def delete_value(self, data):
    if self.head is None:
        print("Linked list is empty")
        return

    if self.head.data == data:
        self.delete_first()
        return

    temp = self.head

    while temp.next != self.head:
        if temp.next.data == data:
            temp.next = temp.next.next
            return
```

```
            temp = temp.next

        print("Value not found")

    # 6. Delete first node
    def delete_first(self):
        if self.head is None:
            print("Linked list is empty")
            return

        if self.head.next == self.head:
            self.head = None
            return

        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        self.head = self.head.next
        temp.next = self.head

    # 7. Delete last node
    def delete_last(self):
        if self.head is None:
            print("Linked list is empty")
            return

        if self.head.next == self.head:
```

```
        self.head = None
        return

    temp = self.head

    while temp.next.next != self.head:
        temp = temp.next

    temp.next = self.head

# 8. Count number of nodes
def count(self):
    if self.head is None:
        print("Number of nodes: 0")
        return

    count = 0
    temp = self.head

    while True:
        count = count + 1
        temp = temp.next

        if temp == self.head:
            break

    print("Number of nodes:", count)

# 9. Display / Traverse
```

```
    def display(self):
        if self.head is None:
            print("Linked list is empty")
            return

        temp = self.head

        while True:
            print(temp.data, end=" -> ")
            temp = temp.next

            if temp == self.head:
                break

        print("(back to head)")


l = CircularLinkedList()

l.create()
print("Linked list:", end=" ")
l.display()

data = int(input("Enter data to insert at beginning: "))
l.insert_begin(data)
print("After insertion:", end=" ")
l.display()

data = int(input("Enter data to insert at end: "))
```

```
l.insert_end(data)
print("After insertion:", end=" ")
l.display()

index = int(input("Enter index: "))
data = int(input("Enter data: "))
l.insert_index(index, data)
print("After insertion:", end=" ")
l.display()

data = int(input("Enter value to delete: "))
l.delete_value(data)
print("After deletion:", end=" ")
l.display()

l.delete_first()
print("After deleting first node:", end=" ")
l.display()

l.delete_last()
print("After deleting last node:", end=" ")
l.display()

l.count()
```

OUTPUT:

```
>>>
========== RESTART: C:/Users/mohit/OneDrive/Desktop/lab4_3.py ==========
Enter number of elements: 5
Enter data: 31
Enter data: 98
Enter data: 47
Enter data: 86
Enter data: 34
Linked list: 31 -> 98 -> 47 -> 86 -> 34 -> (back to head)
Enter data to insert at beginning: 31
After insertion: 31 -> 31 -> 98 -> 47 -> 86 -> 34 -> (back to head)
Enter data to insert at end: 47
After insertion: 31 -> 31 -> 98 -> 47 -> 86 -> 34 -> 47 -> (back to head)
Enter index: 4
Enter data: 98
After insertion: 31 -> 31 -> 98 -> 47 -> 98 -> 86 -> 34 -> 47 -> (back to head)
Enter value to delete: 86
After deletion: 31 -> 31 -> 98 -> 47 -> 98 -> 34 -> 47 -> (back to head)
After deleting first node: 31 -> 98 -> 47 -> 98 -> 34 -> 47 -> (back to head)
After deleting last node: 31 -> 98 -> 47 -> 98 -> 34 -> (back to head)
Number of nodes: 5
>>>
```

EXPLANATION:

The CircularLinkedList program performs various operations such as creating, inserting, deleting, counting, and displaying elements in a circular linked list.

The create() function creates the circular linked list, while insert_begin(), insert_end(), and insert_index() insert new nodes at different positions.

The delete_value(), delete_first(), and delete_last() functions remove nodes from the list. The count() function counts the number of nodes, and display() traverses and displays all the elements.

In a circular linked list, the last node always points back to the first node (head).

ERROR TABLE:

S NO	ERRORS	RECTIFICATION
1	ValueError occurs if a non-integer value is entered when int(input()) is used.	Enter only integer values for the number of elements, index, and node data.
2	IndentationError may occur if the statements inside classes, functions, if, else, while, or for blocks are not properly indented.	Maintain proper indentation throughout the program.
3	AttributeError may occur if an incorrect attribute such as data or next is used.	Make sure self.data and self.next are used correctly.

WEEK – 5

5.AIM: Write a Python program to implement a Stack using an Array and a Linked List. Implement stack operations such as push, pop, peek, and display the stack elements.

a.Stack Using Array:

PROGRAM:

```
class StackArray:
    def __init__(self):
        self.stack = []

    # Push operation
    def push(self, data):
        self.stack.append(data)
        print(data, "pushed into stack")

    # Pop operation
    def pop(self):
        if not self.stack:
            print("Stack Underflow")
        else:
            print(self.stack.pop(), "popped from stack")

    # Peek operation
    def peek(self):
        if not self.stack:
            print("Stack is empty")
        else:
            print("Top element:", self.stack[-1])
```

```
    # Display operation
    def display(self):
        if not self.stack:
            print("Stack is empty")
        else:
            print("Stack elements:", self.stack[::-1])



s = StackArray()

n = int(input("Enter number of elements: "))

for i in range(n):
    data = int(input("Enter data: "))
    s.push(data)

print("Stack:")
s.display()

s.peek()

data = int(input("Enter data to push: "))
s.push(data)

print("After push:")
s.display()

s.pop()
```

```
print("After pop:")
s.display()


s.peek()
```

OUTPUT:

```
>>>
========== RESTART: C:/Users/mohit/OneDrive/Desktop/lab5_2.py ==========
Enter number of elements: 4
Enter data: 17
17 pushed into stack
Enter data: 12
12 pushed into stack
Enter data: 26
26 pushed into stack
Enter data: 16
16 pushed into stack
Stack:
Stack elements: [16, 26, 12, 17]
Top element: 16
Enter data to push: 6
6 pushed into stack
After push:
Stack elements: [6, 16, 26, 12, 17]
6 popped from stack
After pop:
Stack elements: [16, 26, 12, 17]
Top element: 16
>>>
```

b.Stack Using Linked List:

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None



class StackLinkedList:
    def __init__(self):
        self.top = None

    # Push operation
    def push(self, data):
        new = Node(data)
        new.next = self.top
        self.top = new
```

```
    print(data, "pushed into stack")

# Pop operation
def pop(self):
    if self.top is None:
        print("Stack Underflow")
    else:
        print(self.top.data, "popped from stack")
        self.top = self.top.next

# Peek operation
def peek(self):
    if self.top is None:
        print("Stack is empty")
    else:
        print("Top element:", self.top.data)

# Display operation
def display(self):
    if self.top is None:
        print("Stack is empty")
        return

    temp = self.top

    print("Stack elements:", end=" ")

    while temp:
        print(temp.data, end=" ")
```

```
            temp = temp.next

        print()



s = StackLinkedList()

n = int(input("Enter number of elements: "))

for i in range(n):
    data = int(input("Enter data: "))
    s.push(data)

print("Stack:")
s.display()

s.peek()

data = int(input("Enter data to push: "))
s.push(data)

print("After push:")
s.display()

s.pop()

print("After pop:")
s.display()
```

s.peek()

OUTPUT:

```
>>>
==================== RESTART: C:/Users/mohit/OneDrive/Desktop/lab5_1.py ==========
Enter number of elements: 5
Enter data: 11
11 pushed into stack
Enter data: 7
7 pushed into stack
Enter data: 15
15 pushed into stack
Enter data: 9
9 pushed into stack
Enter data: 12
12 pushed into stack
Stack:
Stack elements: 12 9 15 7 11
Top element: 12
Enter data to push: 4
4 pushed into stack
After push:
Stack elements: 4 12 9 15 7 11
4 popped from stack
After pop:
Stack elements: 12 9 15 7 11
Top element: 12
>>>
```

EXPLANATION:

The program implements a Stack using both an Array and a Linked List.

A stack follows the LIFO (Last In, First Out) principle, where the element inserted last is removed first.

The push() operation adds an element to the top of the stack, while the pop() operation removes the top element. The peek() operation displays the top element without removing it. The display() operation displays all the elements present in the stack.

In the array implementation, Python's list is used to store stack elements. In the linked-list implementation, nodes are created dynamically and the top pointer represents the top of the stack.

S NO	ERRORS	RECTIFICATION
1	Stack Underflow occurs when pop() is performed on an empty stack.	Check whether the stack is empty before performing pop().
2	Stack is empty occurs when peek() or display() is performed without any elements.	Push elements into the stack before performing these operations.

S NO	ERRORS	RECTIFICATION
3	Incorrect top element may be displayed if the top position is not updated properly.	Update the top pointer or array position correctly after every push and pop.

Week - 6

6a**. AIM:** Write a python program to implement queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach implement using array, linked list.

PROGRAM: Using an array

queue using arrays.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/queue using arrays.py (3.14.0)

File Edit Format Run Options Window Help

```
class QueueEx:
    def __init__(self, size=5):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, x):
        if self.rear == self.size - 1:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0

            self.rear += 1
            self.queue[self.rear] = x
            print(f"{x} inserted into the queue")

    def dequeue(self):
        if self.front == -1 or self.front > self.rear:
            print("Queue Underflow")
        else:
            x = self.queue[self.front]
            self.queue[self.front] = None
            self.front += 1
            print(f"{x} deleted from the queue")

            if self.front > self.rear:
                self.front = -1
                self.rear = -1

    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
```

queue using arrays.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/queue using arrays.py (3.14.0)

File Edit Format Run Options Window Help

```
    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            for i in range(self.front, self.rear + 1):
                print(self.queue[i])


size = int(input("Enter the size of Queue: "))
q = QueueEx(size)

while True:
    print("\n----- Queue Menu -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

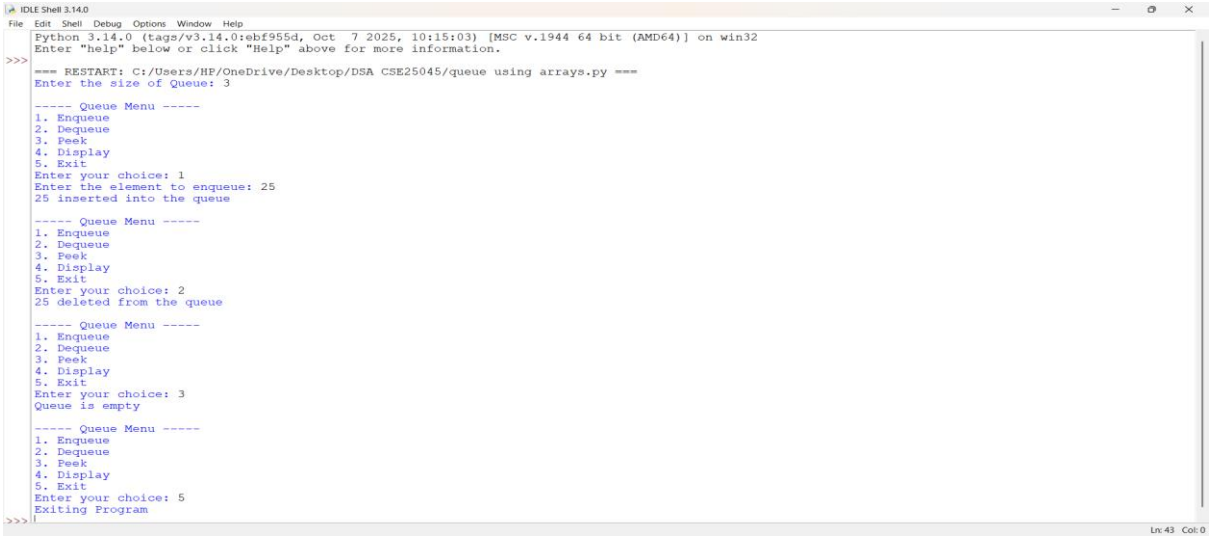
    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Exiting Program")
        break

    else:
        print("Invalid choice")
```

Ln: 27 Col: 12

OUTPUT:



PROGRAM: Using linked list





OUTPUT:



EXPLANATION:

- class QueueEx: Creates a **class** named QueueEx for implementing the queue.
- __init__(self, size=5): **Constructor** that initializes the queue with a default size of 5.
- self.size = size : Stores the user-given queue size.
- self.queue = [None] * size : Creates a list of the given size and initializes all positions with None.
- self.front = -1 : Initializes front to indicate that the queue is empty.
- self.rear = -1 : Initializes rear to indicate that no element is present.

enqueue()

- if self.rear == self.size - 1: Checks whether rear has reached the last position.
- self.front = 0 : Sets the front position when inserting the first element.
- self.rear += 1 : Increases rear to the next position.
- self.queue[self.rear] = x : Stores x at the rear position.
- print() : Displays the insertion message.

dequeue()

- if self.front == -1 or self.front > self.rear: Checks whether the queue is empty.
- x = self.queue[self.front] : Stores the front element in x.
- self.queue[self.front] = None : Removes the front element from the list.
- self.front += 1 : Moves front to the next position.
- if self.front > self.rear: : Checks whether all elements have been deleted.
- self.front = -1 and self.rear = -1 : Resets the queue to the empty state.

peek()

- if self.front == -1: Checks whether the queue is empty.
- self.queue[self.front] : Accesses and displays the front element without deleting it.

display()

- if self.front == -1: Checks whether the queue is empty.
- range(self.front, self.rear + 1) : Traverses the queue from front to rear.
- print(self.queue[i]) : Displays each queue element.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Queue Overflow occurs when the queue is full.	Check if self.rear == self.size - 1 before inserting an element.
2.	Queue Underflow occurs when the queue is empty.	Check if self.front == -1 or self.front > self.rear before deleting an element.

6b**. AIM:** Write a python program to implement queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach implement using circular linked list.

PROGRAM:



```
            self.count -= 1
            print(f"{x} deleted from the queue")

    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:", self.front.data)

    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            temp = self.front
            while True:
                print(temp.data)
                temp = temp.next

                if temp == self.front:
                    break

size = int(input("Enter the size of Queue: "))
q = CircularQueue(size)
while True:
    print("\n----- Queue Menu -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()
```

```
    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Exiting Program")
        break

    else:
        print("Invalid Choice!")
```

OUTPUT:

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/HP/OneDrive/Desktop/DSA CSE25045/queue using cll.py ====
Enter the size of Queue: 3

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 45
45 inserted into the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 25
25 inserted into the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
45 deleted from the queue
```

```
----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 25

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Exiting Program
>>>
```

EXPLANATION:

1. Node Linking

- self.next = None : Initializes the node link as empty.
- new_node = Node(x) : Creates a new node with the given data.

- new_node.next = new_node : Makes the first node point to itself, forming a circular link.

2. Circular Connection During Enqueue

- new_node.next = self.front : Connects the new node to the first node.
- self.rear.next = new_node : Connects the current last node to the new node.
- self.rear = new_node : Updates the last node to the newly inserted node.

3. Circular Connection During Dequeue

- self.front = self.front.next : Moves front to the next node.
- self.rear.next = self.front : Reconnects the last node to the new front node.
- if self.front == self.rear: : Checks whether only one node exists.
- self.front = None : Removes the only node by clearing the front reference.
- self.rear = None : Clears the rear reference.

4. Circular Traversal

- temp = self.front : Starts traversal from the first node.
- temp = temp.next : Moves to the next linked node.
- if temp == self.front: : Detects that traversal has returned to the first node.
- break : Stops traversal after one complete circular cycle.

5. Node Data Access

- self.front.data : Accesses the data stored in the front node.
- temp.data : Accesses the data of the current node during traversal.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	new_node.next is not connected to front, breaking the circular link.	Set new_node.next = self.front.
2.	self.rear.next is not updated after insertion or deletion.	Update self.rear.next = new_node during enqueue and self.rear.next = self.front during dequeue.

WEEK – 7

7**. AIM:** Write a python program to implement binary tree using an array and a linked list with all of its operation.

PROGRAM: Using an array

Binary tree using an array.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/Binary tree using an array.py (3.14.0)

File Edit Format Run Options Window Help

```
class BinaryTree:
    def __init__(self, size):
        self.tree = [None] * size
        self.size = size

    def insert(self, value):
        for i in range(self.size):
            if self.tree[i] is None:
                self.tree[i] = value
                print(value, "inserted")
                return
        print("Tree is full")

    def search(self, value):
        for i in range(self.size):
            if self.tree[i] == value:
                print(value, "found at index", i)
                return
        print(value, "not found")

    def delete(self, value):
        for i in range(self.size):
            if self.tree[i] == value:
                self.tree[i] = None
                print(value, "deleted")
                return
        print(value, "not found")

    def inorder(self, i=0):
        if i < self.size and self.tree[i] is not None:
            self.inorder(2 * i + 1)
            print(self.tree[i], end=" ")
            self.inorder(2 * i + 2)

    def preorder(self, i=0):
        if i < self.size and self.tree[i] is not None:
            print(self.tree[i], end=" ")
            self.preorder(2 * i + 1)
            self.preorder(2 * i + 2)
```

Binary tree using an array.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/Binary tree using an array.py (3.14.0)

File Edit Format Run Options Window Help

```
        if i < self.size and self.tree[i] is not None:
            print(self.tree[i], end=" ")
            self.preorder(2 * i + 1)
            self.preorder(2 * i + 2)

    def postorder(self, i=0):
        if i < self.size and self.tree[i] is not None:
            self.postorder(2 * i + 1)
            self.postorder(2 * i + 2)
            print(self.tree[i], end=" ")

bt = BinaryTree(10)

while True:
    print("\n1.Insert  2.Delete  3.Search")
    print("4.Inorder  5.Preorder  6.Postorder  7.Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        bt.insert(value)
    elif choice == 2:
        value = int(input("Enter value to delete: "))
        bt.delete(value)
    elif choice == 3:
        value = int(input("Enter value to search: "))
        bt.search(value)
    elif choice == 4:
        bt.inorder()
        print()
    elif choice == 5:
        bt.preorder()
        print()
    elif choice == 6:
        bt.postorder()
        print()
    elif choice == 7:
        break
    else:
        print("Invalid choice")
```

Ln: 47 Col: 0

OUTPUT:

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DSA CSE25045/Binary tree using an array.py

1.Insert  2.Delete  3.Search
4.Inorder  5.Preorder  6.Postorder  7.Exit
Enter choice: 1
Enter value: 10
10 inserted

1.Insert  2.Delete  3.Search
4.Inorder  5.Preorder  6.Postorder  7.Exit
Enter choice: 1
Enter value: 30
30 inserted

1.Insert  2.Delete  3.Search
4.Inorder  5.Preorder  6.Postorder  7.Exit
Enter choice: 4
30 10

1.Insert  2.Delete  3.Search
4.Inorder  5.Preorder  6.Postorder  7.Exit
Enter choice: 6
30 10

1.Insert  2.Delete  3.Search
4.Inorder  5.Preorder  6.Postorder  7.Exit
Enter choice: 7
>>>
```

PROGRAM: Using linked list

Binary tree using linked list.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/Binary tree using linked list.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BinaryTree:
    def __init__(self):
        self.root = None

    def insert(self, data):
        new_node = Node(data)

        if self.root is None:
            self.root = new_node
            print(data, "inserted")
            return
        queue = [self.root]

        while queue:
            current = queue.pop(0)

            if current.left is None:
                current.left = new_node
                print(data, "inserted")
                return
            else:
                queue.append(current.left)

            if current.right is None:
                current.right = new_node
                print(data, "inserted")
                return
            else:
                queue.append(current.right)

    def search(self, data):
        if self.root is None:
            print("Tree is empty")
            return

        queue = [self.root]
```

Ln: 120 Col: 0

Binary tree using linked list.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/Binary tree using linked list.py (3.14.0)

File Edit Format Run Options Window Help

```
        while queue:
            current = queue.pop(0)

            if current.data == data:
                print(data, "found")
                return

            if current.left:
                queue.append(current.left)

            if current.right:
                queue.append(current.right)

        print(data, "not found")

    def delete(self, data):
        if self.root is None:
            print("Tree is empty")
            return
        queue = [self.root]
        target = None
        deepest = None
        parent = None
        deepest_parent = None

        while queue:
            current = queue.pop(0)

            if current.data == data:
                target = current

            if current.left:
                parent = current
                queue.append(current.left)
                deepest_parent = current

            if current.right:
                parent = current
                queue.append(current.right)
                deepest_parent = current

            deepest = current
```

Ln: 120 Col: 0

Binary tree using linked list.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/Binary tree using linked list.py (3.14.0)

File Edit Format Run Options Window Help

```
        if target is None:
            print(data, "not found")
            return

        if deepest == self.root:
            self.root = None
        else:
            if deepest_parent.left == deepest:
                deepest_parent.left = None
            else:
                deepest_parent.right = None
            target.data = deepest.data
        print(data, "deleted")

    def inorder(self, node):
        if node:
            self.inorder(node.left)
            print(node.data, end=" ")
            self.inorder(node.right)

    def preorder(self, node):
        if node:
            print(node.data, end=" ")
            self.preorder(node.left)
            self.preorder(node.right)

    def postorder(self, node):
        if node:
            self.postorder(node.left)
            self.postorder(node.right)
            print(node.data, end=" ")

bt = BinaryTree()
while True:
    print("\n1.Insert  2.Delete  3.Search")
    print("4.Inorder  5.Preorder  6.Postorder  7.Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        bt.insert(value)
```

Ln: 119 Col: 17

```
    elif choice == 2:
        value = int(input("Enter value to delete: "))
        bt.delete(value)

    elif choice == 3:
        value = int(input("Enter value to search: "))
        bt.search(value)

    elif choice == 4:
        bt.inorder(bt.root)
        print()

    elif choice == 5:
        bt.preorder(bt.root)
        print()

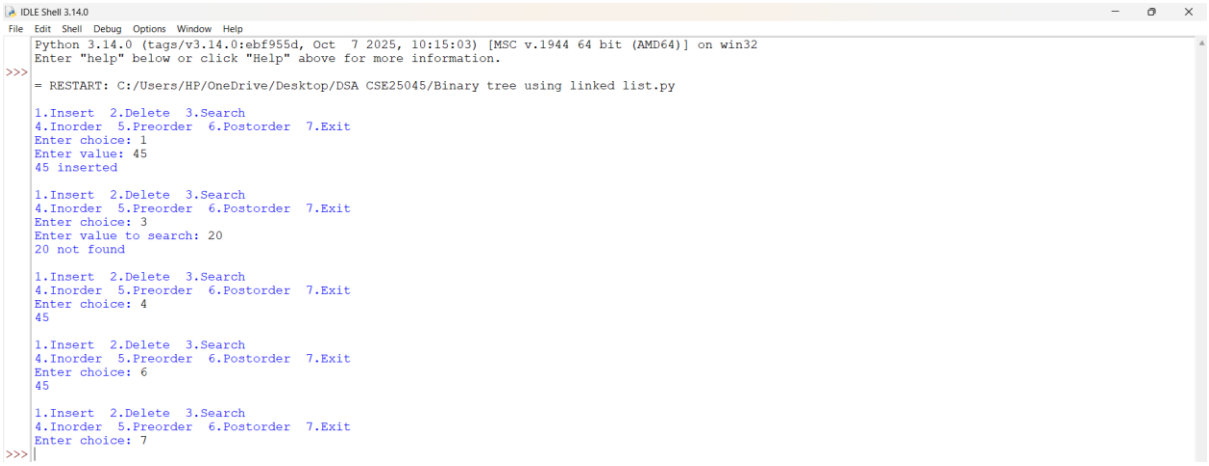
    elif choice == 6:
        bt.postorder(bt.root)
        print()

    elif choice == 7:
        break

    else:
        print("Invalid choice")
```

Ln: 119 Col: 17

OUTPUT:



EXPLANATION:

Binary Tree Using Array

- class BinaryTree: Defines the binary tree class.
- __init__(self, size): Creates a fixed-size array filled with None.
- insert(value): Inserts a value into the first empty array position.
- search(value): Searches for a value and displays its index.
- delete(value): Deletes a value by replacing its array position with None.
- inorder(i=0): Traverses Left → Root → Right using array indices.
- preorder(i=0): Traverses Root → Left → Right.
- postorder(i=0): Traverses Left → Right → Root.
- bt = BinaryTree(10): Creates an array-based tree with capacity 10.
- Child indices: Left child = 2*i + 1; right child = 2*i + 2.

Binary Tree Using Linked Representation

- class Node: Defines a node containing data and left and right references.
- __init__(self, data): Initializes the node's data and sets both child references to None.

- __init__(self): Initializes an empty tree with self.root = None.
- insert(data): Creates a node and uses a queue to insert it at the first available position in level order.
- search(data): Uses a queue to search nodes level by level.
- delete(data): Replaces the target's data with the deepest node's data and removes the deepest node. The supplied code has a bug in tracking the deepest node's parent.
- inorder(node), preorder(node), postorder(node): Perform the three traversals described in Code 1, using node references instead of array indices.
- bt = BinaryTree(): Creates an empty linked binary tree.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Array-based deletion sets a node to None, making its existing children inaccessible during traversal. Ensure deletion preserves the tree structure or use proper deepest-node replacement.	Ensure deletion preserves the tree structure or use proper deepest-node replacement.
2.	Linked-tree deletion tracks deepest_parent incorrectly.	Update the parent reference whenever a child is visited, and track the actual parent of the deepest node.
3.	parent variable in the linked-tree deletion method is assigned but never used.	Remove the unused variable or use it to track the deepest node's parent correctly.

WEEK – 8

8**. AIM:** Write a python program to implement binary search trees with its operation.

PROGRAM:



BST.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/BST.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BST:
    def __init__(self):
        self.root = None

    def insert(self, root, data):
        if root is None:
            return Node(data)
        if data < root.data:
            root.left = self.insert(root.left, data)
        elif data > root.data:
            root.right = self.insert(root.right, data)

        return root

    def search(self, root, key):
        if root is None or root.data == key:
            return root

        if key < root.data:
            return self.search(root.left, key)
        else:
            return self.search(root.right, key)

    def find_min(self, root):
        while root.left is not None:
            root = root.left
        return root

    def delete(self, root, key):
        if root is None:
            return root

        if key < root.data:
            root.left = self.delete(root.left, key)
        elif key > root.data:
            root.right = self.delete(root.right, key)
```

```
*BST.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/BST.py (3.14.0)*
File  Edit  Format  Run  Options  Window  Help
        elif key > root.data:
            root.right = self.delete(root.right, key)
        else:
            if root.left is None:
                return root.right
            elif root.right is None:
                return root.left

            temp = self.find_min(root.right)
            root.data = temp.data
            root.right = self.delete(root.right, temp.data)

        return root

    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.data, end=" ")
            self.inorder(root.right)

    def preorder(self, root):
        if root:
            print(root.data, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.data, end=" ")

tree = BST()

while True:
    print("\n\n1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Exit")
                                                                    Ln: 19  Col: 8
```

```
*BST.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/BST.py (3.14.0)*
File  Edit  Format  Run  Options  Window  Help
    choice = int(input("Enter your choice: "))
    if choice == 1:
        data = int(input("Enter value to insert: "))
        tree.root = tree.insert(tree.root, data)
        print("Node inserted successfully.")

    elif choice == 2:
        key = int(input("Enter value to search: "))
        result = tree.search(tree.root, key)

        if result:
            print("Element found.")
        else:
            print("Element not found.")

    elif choice == 3:
        key = int(input("Enter value to delete: "))
        if tree.search(tree.root, key):
            tree.root = tree.delete(tree.root, key)
            print("Node deleted successfully.")
        else:
            print("Element not found.")

    elif choice == 4:
        print("Inorder Traversal:", end=" ")
        tree.inorder(tree.root)

    elif choice == 5:
        print("Preorder Traversal:", end=" ")
        tree.preorder(tree.root)

    elif choice == 6:
        print("Postorder Traversal:", end=" ")
        tree.postorder(tree.root)

    elif choice == 7:
        print("Program terminated.")
        break

    else:
        print("Invalid choice. Please try again.")
                                                                    Ln: 124  Col: 50
```

OUTPUT:

```
IDLE Shell 3.14.0
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/HP/OneDrive/Desktop/DSA CSE25045/BST.py ==========


1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 1
Enter value to insert: 87
Node inserted successfully.


1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 1
Enter value to insert: 23
Node inserted successfully.


1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 6
Postorder Traversal: 23 87
```

```
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 7
Program terminated.
>>>
                                                                    Ln: 50  Col: 0
```

EXPLANATION:

BST Initialization

- class BST: Defines the BST class.
- self.root = None: Initializes the root.

Insertion

- insert(root, data): Inserts a value according to BST ordering.
- root is None: Creates a new node.
- data < root.data: Recursively inserts into the left subtree.
- data > root.data: Recursively inserts into the right subtree.
- return root: Returns the updated subtree root.

Searching

- search(root, key): Searches recursively for the specified key.
- root is None or root.data == key: Returns the node or None.
- key < root.data: Searches the left subtree; otherwise, searches the right subtree.

Finding Minimum

- find_min(root): Moves left until the leftmost node is reached.
- return root: Returns the minimum-valued node.

Deletion

- delete(root, key): Deletes a node while preserving BST ordering.
- Handles nodes with zero or one child by returning the appropriate child.
- For two children, finds the inorder successor, copies its value, and deletes the successor.

Main Program

- tree = BST(): Creates a BST object.
- tree.root = tree.insert(...) and tree.root = tree.delete(...): Update the root after modifications.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Incorrect root assignment after insertion or deletion may cause	Assign the returned node to tree.root using tree.root =

	the updated tree structure to be lost.	tree.insert(tree.root, data) and tree.root = tree.delete(tree.root, key).
2.	Deleting a node with two children may violate BST ordering if the replacement node is not selected correctly.	Use find_min(root.right) to find the inorder successor and delete that successor from the right subtree.

WEEK – 9

9**. AIM:** Write a python program to implement AVL trees with its operation.

PROGRAM:



```
*AVL TREE.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/AVL TREE.py (3.14.0)*
File  Edit  Format  Run  Options  Window  Help
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1

class AVLTree:
    def height(self, root):
        if root is None:
            return 0
        return root.height

    def get_balance(self, root):
        if root is None:
            return 0
        return self.height(root.left) - self.height(root.right)

    def right_rotate(self, y):
        x = y.left
        T2 = x.right
        x.right = y
        y.left = T2

        y.height = 1 + max(self.height(y.left),
                           self.height(y.right))
        x.height = 1 + max(self.height(x.left),
                           self.height(x.right))
        return x

    def left_rotate(self, x):
        y = x.right
        T2 = y.left

        y.left = x
        x.right = T2

        x.height = 1 + max(self.height(x.left),
                           self.height(x.right))
        y.height = 1 + max(self.height(y.left),
                           self.height(y.right))
        return y
Ln: 21  Col: 20
```



```
*AVL TREE.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/AVL TREE.py (3.14.0)*
File  Edit  Format  Run  Options  Window  Help
    def insert(self, root, key):
        if root is None:
            return Node(key)

        if key < root.key:
            root.left = self.insert(root.left, key)
        elif key > root.key:
            root.right = self.insert(root.right, key)
        else:
            return root

        root.height = 1 + max(self.height(root.left),
                              self.height(root.right))
        balance = self.get_balance(root)

        if balance > 1 and key < root.left.key:
            return self.right_rotate(root)

        if balance < -1 and key > root.right.key:
            return self.left_rotate(root)

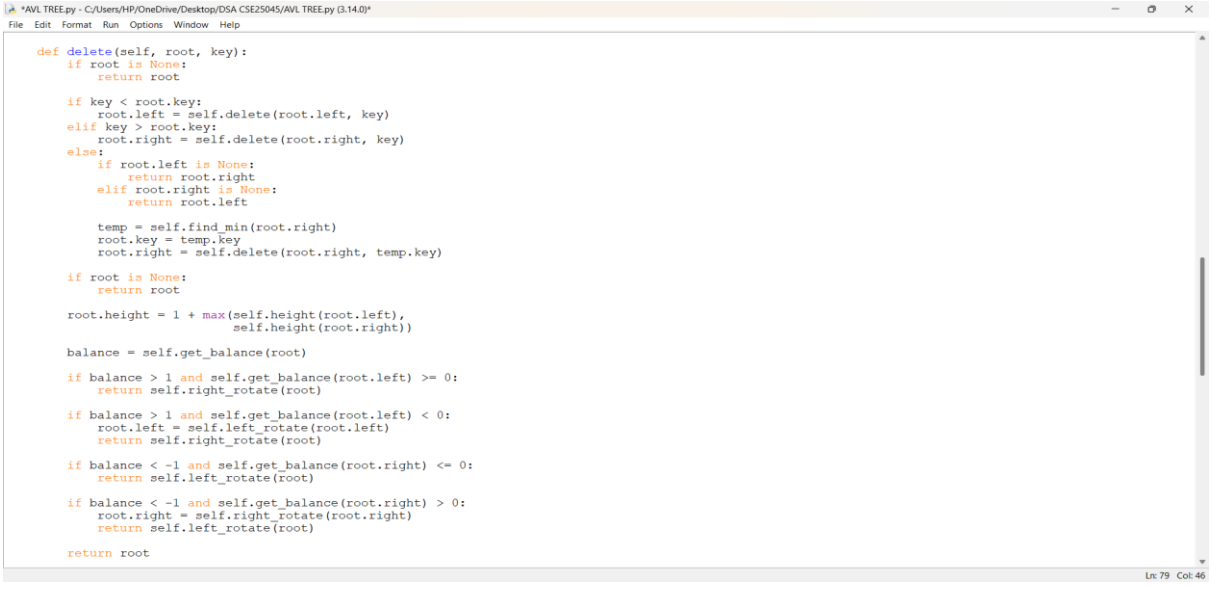
        if balance > 1 and key > root.left.key:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        if balance < -1 and key < root.right.key:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)
        return root

    def search(self, root, key):
        if root is None or root.key == key:
            return root

        if key < root.key:
            return self.search(root.left, key)
        return self.search(root.right, key)

    def find_min(self, root):
        while root.left is not None:
            root = root.left
        return root
Ln: 79  Col: 46
```



```
*AVL TREE.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/AVL TREE.py (3.14.0)*
File  Edit  Format  Run  Options  Window  Help
    def delete(self, root, key):
        if root is None:
            return root

        if key < root.key:
            root.left = self.delete(root.left, key)
        elif key > root.key:
            root.right = self.delete(root.right, key)
        else:
            if root.left is None:
                return root.right
            elif root.right is None:
                return root.left

            temp = self.find_min(root.right)
            root.key = temp.key
            root.right = self.delete(root.right, temp.key)

        if root is None:
            return root

        root.height = 1 + max(self.height(root.left),
                              self.height(root.right))

        balance = self.get_balance(root)

        if balance > 1 and self.get_balance(root.left) >= 0:
            return self.right_rotate(root)

        if balance > 1 and self.get_balance(root.left) < 0:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        if balance < -1 and self.get_balance(root.right) <= 0:
            return self.left_rotate(root)

        if balance < -1 and self.get_balance(root.right) > 0:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root
Ln: 79  Col: 46
```

```
*AVL TREE.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/AVL TREE.py (3.14.0)*
File  Edit  Format  Run  Options  Window  Help
    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.key, end=" ")
            self.inorder(root.right)

    def preorder(self, root):
        if root:
            print(root.key, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.key, end=" ")

tree = AVLTree()
root = None
while True:
    print("\n\n1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        key = int(input("Enter value to insert: "))
        root = tree.insert(root, key)
        print("Node inserted successfully.")

    elif choice == 2:
        key = int(input("Enter value to search: "))
        if tree.search(root, key):
            print("Element found.")
        else:
            print("Element not found.")
Ln: 148  Col: 11
```

```
    elif choice == 3:
        key = int(input("Enter value to delete: "))
        if tree.search(root, key):
            root = tree.delete(root, key)
            print("Node deleted successfully.")
        else:
            print("Element not found.")

    elif choice == 4:
        print("Inorder Traversal:", end=" ")
        tree.inorder(root)

    elif choice == 5:
        print("Preorder Traversal:", end=" ")
        tree.preorder(root)

    elif choice == 6:
        print("Postorder Traversal:", end=" ")
        tree.postorder(root)

    elif choice == 7:
        print("Program terminated.")
        break

    else:
        print("Invalid choice.")
Ln: 148  Col: 11
```

OUTPUT:

```
IDLE Shell 3.14.0
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
======== RESTART: C:/Users/HP/OneDrive/Desktop/DSA CSE25045/AVL TREE.py ========


1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 1
Enter value to insert: 98
Node inserted successfully.


1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 1
Enter value to insert: 44
Node inserted successfully.


1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 5
Preorder Traversal: 98 44
```

```
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 2
Enter value to search: 44
Element found.


1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 7
Program terminated.
>>>
Ln: 62  Col: 0
```

EXPLANATION:

1. Node Class

- self.height = 1: Stores the height of each node, which is required for balancing.

2. Height Calculation

- height(root): Returns the height of a node; returns 0 for None.
- get_balance(root): Calculates the balance factor as height(root.left) - height(root.right).

3. Rotations

- right_rotate(y): Performs a right rotation to rebalance a subtree.
- left_rotate(x): Performs a left rotation to rebalance a subtree.
- T2: Temporarily stores a subtree during rotation.
- Node heights are recalculated after each rotation.
- The rotations handle LL, RR, LR, and RL imbalance cases.

4. Insertion

- root.height = 1 + max(...): Updates the height after insertion.
- balance = self.get_balance(root): Checks whether the subtree is unbalanced.
- The four conditional statements identify the LL, RR, LR, and RL cases and apply the required rotations.

5. Deletion

- Recalculates node heights after deletion.
- Checks the updated balance factor.
- Uses the child node's balance factor to select the appropriate single or double rotation.
- Returns the new subtree root after rebalancing.

6. Main Program

- root = tree.insert(root, key): Updates the root after insertion and rebalancing.
- root = tree.delete(root, key): Updates the root after deletion and rebalancing.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Incorrect height update after insertion or deletion may cause an incorrect balance factor.	Update the node height using 1 + max(height(left), height(right)) after modifying the subtree.

2.	Incorrect rotation selection can leave the AVL tree unbalanced.	Check the balance factor and apply the correct LL, RR, LR, or RL rotation during insertion and deletion.